RLC (Random Linear Combination) 기법

개요

RLC(Random Linear Combination, 무작위 선형 결합)는 ZKEVM 구현체에서 데이터 압축과 효율적인 검증을 위해 사용되는 핵심 암호학적 기법입니다. RLC는 여러 바이트나 필드 원소를 하나의 필드 원소로 압축하여, 다중 컬럼 룩업과 데이터 무결성 검증을 효율적으로 수행할 수 있게 합니다.

RLC의 목적

1. **데이터 압축**: 여러 바이트를 하나의 필드 원소로 결합
2. **다중 컬럼 룩업**: 여러 컬럼을 하나의 값으로 압축하여 룩업 효율성 향상
3. **무결성 검증**: 높은 확률로 데이터 일치 여부 검증
4. **회로 크기 감소**: 제약 조건 수를 줄여 증명 크기 최소화

기본 원리

RLC는 다음과 같은 수식으로 정의됩니다:

$$RLC(v_0, v_1, ..., v_{n-1}) = v_0 + v_1 \cdot r + v_2 \cdot r^2 + ... + v_{n-1} \cdot r^{n-1}$$

여기서:

- v_i : 입력 값들 (바이트, 필드 원소 등)
- r : 무작위 challenge 값
- 결과는 하나의 필드 원소

파일: `zkevm-circuits/src/evm_circuit/util.rs`

```rust
pub(crate) mod rlc {
use  crate::util::{Expr, Field};
use halo2_proofs::plonk::Expression;


// Expression 버전: 제약 조건 구성용
pub(crate) fn  expr<F: Field, E: Expr<F>>(
expressions: &[E],
randomness: E
) -> Expression<F> {
if !expressions.is_empty() {
generic(
expressions.iter().map(|e| e.expr()),
randomness.expr()
)
} else {
0.expr()
}
}


// Value 버전: witness 할당용
pub(crate) fn  value<'a, F: Field, I>(values: I, randomness: F) -> F
where
```

```
I: IntoIterator<Item = &'a u8>,
<I as IntoIterator>::IntoIter: DoubleEndedIterator,
{
let values = values
.into_iter()
.map(|v| F::from(*v as u64))
.collect::<Vec<F>>();
if !values.is_empty() {
generic(values, randomness)
} else {
F::zero()
}
}


// 제네릭 구현: Horner's method 사용
fn generic<V, I>(values: I, randomness: V) -> V
where
I: IntoIterator<Item = V>,
<I as IntoIterator>::IntoIter: DoubleEndedIterator,
V: Clone + Add<Output = V> + Mul<Output = V>,
{
let mut values = values.into_iter().rev();
let init = values.next().expect("values should not be empty");
```

```
// Horner's method: ((v_{n-1} * r + v_{n-2} ) * r + ...) * r + v_0
values.fold(init, |acc, value| acc * randomness.clone() + value)
}
}
```

Challenge 시스템

Challenge 구조

ZKEVM은 세 가지 주요 challenge를 사용합니다:

파일: `zkevm-circuits/src/util.rs`

```rust
#[derive(Default, Clone, Copy, Debug)]
pub struct Challenges<T = Challenge> {
evm_word: T, // EVM 워드(32바이트) 인코딩용
keccak_input: T, // Keccak 입력 데이터 인코딩용
lookup_input: Option<T>, // 룩업 테이블 인코딩용 (SuperCircuit만)
}
```

Challenge 생성

Challenge는 Halo2의 phase 시스템을 통해 생성됩니다:

```rust
impl Challenges {
// Phase 1까지의 challenge 생성
```

```rust
pub fn construct_p1<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
evm_word: meta.challenge_usable_after(FirstPhase),
keccak_input: meta.challenge_usable_after(FirstPhase),
lookup_input: None,
}
}

// 모든 phase의 challenge 생성 (SuperCircuit용)
pub fn construct<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
evm_word: meta.challenge_usable_after(FirstPhase),
keccak_input: meta.challenge_usable_after(FirstPhase),
lookup_input: Some(meta.challenge_usable_after(SecondPhase)),
}
}
}
```

Phase 시스템:

- **FirstPhase**: 기본 제약 조건 구성
- **SecondPhase**: `evm_word`, `keccak_input` challenge 사용 가능
- **ThirdPhase**: `lookup_input` challenge 사용 가능 (SuperCircuit만)

Challenge 사용

Challenge는 두 가지 형태로 사용됩니다:

1. Expression 형태 (제약 조건용)

```
pub fn exprs<F: Field>(&self, meta: &mut ConstraintSystem<F>)
-> Challenges<Expression<F>>
{
let [evm_word, keccak_input] = query_expression(meta, |meta| {
[self.evm_word, self.keccak_input]
.map(|challenge| meta.query_challenge(challenge))
});
Challenges {
evm_word,
keccak_input,
lookup_input: self.lookup_input
.map(|c| query_expression(meta, |meta| meta.query_challenge(c))),
}
}
```

2. Value 형태 (witness 할당용)

```
pub fn values<F: Field>(&self, layouter: &impl Layouter<F>)
-> Challenges<Value<F>>
{
Challenges {
```

```
evm_word: layouter.get_challenge(self.evm_word),

keccak_input: layouter.get_challenge(self.keccak_input),

lookup_input: self.lookup_input

.map(|c| layouter.get_challenge(c)),

}

}
```

RLC 사용 사례

1. EVM Word 인코딩

EVM의 32바이트 워드를 하나의 필드 원소로 압축합니다.

파일: `zkevm-circuits/src/evm_circuit/util.rs`

```
pub(crate) type Word<F> = RandomLinearCombination<F, 32>;


#[derive(Clone, Debug)]

pub(crate) struct RandomLinearCombination<F, const N: usize> {

expression: Expression<F>, // RLC 표현식

pub(crate) cells: [Cell<F>; N], // 개별 바이트 셀들

}


impl<F: Field, const N: usize> RandomLinearCombination<F, N> {

pub(crate) fn  new(cells: [Cell<F>; N], randomness: Expression<F>) -> Self {
```

```
Self {
expression: rlc::expr(
&cells.clone().map(|cell| cell.expr()),
randomness
),
cells,
}
}
}
```

사용 예시:

```
// CachedRegion에서 word RLC 계산
pub fn word_rlc(&self, n: U256) -> Value<F> {
self.challenges
.evm_word()
.map(|r| rlc::value(&n.to_le_bytes(), r))
}
```

2. Keccak 입력 인코딩

Keccak 해시 함수의 입력 데이터를 RLC로 인코딩합니다.

파일: `zkevm-circuits/src/table.rs`

```
// Keccak 테이블 엔트리 생성
```

```
pub fn assignments<F: Field>(
input: &[u8],
challenges: &Challenges<Value<F>>,
) -> Vec<[Value<F>; 4]> {
let input_rlc = challenges
.keccak_input()
.map(|challenge| rlc::value(input.iter().rev(), challenge));
let input_len = F::from(input.len() as u64);
let output = keccak256(input);
let output_rlc = challenges
.keccak_input()
.map(|challenge| rlc::value(
&Word::from_big_endian(output).to_le_bytes(),
challenge
));
vec![[
Value::known(F::one()), // is_enabled
Value::known(F::one()), // is_final
input_rlc, // input RLC
Value::known(input_len), // input length
output_rlc, // output RLC
]]
```

```
}
```

제약 조건에서의 사용:

파일: `zkevm-circuits/src/keccak_circuit.rs`

```
// Keccak Circuit에서 데이터 RLC 업데이트
let new_data_rlc = select::expr(
is_padding.expr(),
new_data_rlc.clone(),
new_data_rlc.clone() * challenges.keccak_input() + byte.expr.clone(),
);
```

3. 트랜잭션 데이터 인코딩

트랜잭션의 CallData와 Access List를 RLC로 인코딩합니다.

파일: `zkevm-circuits/src/tx_circuit.rs`

CallData RLC

```
// CallData 바이트를 순차적으로 RLC 계산
let mut rlc = challenges.keccak_input().map(|_| F::zero());

for (idx, byte) in tx.call_data.iter().enumerate() {
rlc = rlc
.zip(challenges.keccak_input())
.map(|(rlc, keccak_input)| {
```

```
rlc * keccak_input + F::from(*byte  as u64)
});
// ... 할당 로직
}
```

Access List RLC

```
// Access List 섹션 RLC 계산
let  mut  section_rlc = challenges.keccak_input().map(|_| F::zero());
let  r20 = challenges.keccak_input().map(|f| f.pow([20, 0, 0, 0]));
let  r32 = challenges.keccak_input().map(|f| f.pow([32, 0, 0, 0]));

for  al  in  tx.access_list.as_ref().unwrap().0.iter() {
// 주소 (20바이트) RLC
let  field_rlc = rlc_be_bytes(
&al.address.to_fixed_bytes(),
challenges.keccak_input()
);
section_rlc = section_rlc * r20 + field_rlc;
// 스토리지 키 (32바이트) RLC
for  sk  in  al.storage_keys.iter() {
let  field_rlc = rlc_be_bytes(
&sk.to_fixed_bytes(),
```

```
challenges.keccak_input()
);
section_rlc = section_rlc * r32 + field_rlc;
}
}
```

제약 조건:

```
// 섹션 RLC 누적 검증
cb.require_equal(
"section_rlc' = section_rlc * r + byte'",
meta.query_advice(section_rlc, Rotation::next()),
meta.query_advice(section_rlc, Rotation::cur())
* challenges.keccak_input()
+ meta.query_advice(tx_table.value, Rotation::next()),
);
```

4. 서명 검증

ECDSA 서명 검증에서 메시지 해시를 RLC로 인코딩합니다.

파일: `zkevm-circuits/src/sig_circuit.rs`

```
// Keccak challenge의 거듭제곱 생성
let  keccak_challenge_powers = iter::successors(
Some(Value::known(F::one())),
```

```
|coeff| Some(challenges.keccak_input() * coeff)
)
.take(64)
.collect::<Vec<_>>();


// 메시지 RLC 계산
let  msg_rlc = challenges
.keccak_input()
.map(|r| rlc::value(sign_data.msg.iter().rev(), r));
```

5. Copy Circuit

Copy Circuit에서 복사되는 데이터의 누적 값을 RLC로 추적합니다.

파일: `zkevm-circuits/src/copy_circuit.rs`

```
// 값 누적 업데이트
let  value_acc_next = select::expr(
is_pad.expr() + is_mask.expr(),
value_acc.expr(),
value_acc.expr() * challenges.keccak_input() + value.expr(),
);
```

6. Public Input Circuit

Public Input Circuit에서 RPI(Rolling Public Input) 바이트를 RLC로 누적합니다.

파일: zkevm-circuits/src/pi_circuit.rs

```
// RLC 누적 제약 조건
let rpi_rlc_acc_cur = meta.query_advice(rpi_rlc_acc, Rotation::cur());
let rpi_bytes_next = meta.query_advice(rpi_bytes, Rotation::next());
let keccak_rand = challenges.keccak_input();
let evm_rand = challenges.evm_word();

// 필드 타입에 따라 다른 challenge 사용
let is_rlc_keccak = meta.query_fixed(is_rlc_keccak, Rotation::next());
let rlc_rand = is_rlc_keccak.clone() * keccak_rand
+ (1.expr() - is_rlc_keccak) * evm_rand;

// RLC 업데이트: acc' = acc * rand + byte
cb.require_equal(
"rpi_rlc_acc update",
rpi_rlc_acc_next,
rpi_rlc_acc_cur * rlc_rand + rpi_bytes_next,
);
```

다중 컬럼 압축

RLC의 주요 용도 중 하나는 여러 컬럼을 하나의 값으로 압축하여 룩업을 효율적으로 만드는 것입니다.

Challenge의 거듭제곱

여러 컬럼을 압축할 때는 challenge의 거듭제곱을 사용합니다:

파일: `zkevm-circuits/src/util.rs`

```
impl<F: Field> Challenges<Expression<F>> {
// Challenge의 거듭제곱 생성
fn powers_of<const S: usize>(base: Expression<F>) -> [Expression<F>; S] {
std::iter::successors(
base.clone().into(),
|power| (base.clone() * power.clone()).into()
)
.take(S)
.collect::<Vec<_>>()
.try_into()
.unwrap()
}

// EVM word용 거듭제곱
pub fn evm_word_powers_of_randomness<const S: usize>(&self)
-> [Expression<F>; S]
{
Self::powers_of(self.evm_word.clone())
}
```

```rust
// Keccak 입력용 거듭제곱
pub fn keccak_powers_of_randomness<const S: usize>(&self)
-> [Expression<F>; S]
{
Self::powers_of(self.keccak_input.clone())
}

// Lookup 입력용 거듭제곱
pub fn lookup_input_powers_of_randomness<const S: usize>(&self)
-> [Expression<F>; S]
{
Self::powers_of(
self.lookup_input
.as_ref()
.expect("created for supercircuit")
.clone()
)
}
}
```

다중 컬럼 룩업 예시

파일: `aggregator/src/aggregation/rlc/config.rs`

```
// Keccak 테이블 룩업 (5개 컬럼)
meta.lookup_any("rlc keccak lookup", |meta| {
let  q = meta.query_selector(lookup_gate_selector);
let  input_rlc = meta.query_advice(phase_2_column, Rotation::cur());
let  output_rlc = meta.query_advice(phase_2_column, Rotation::next());
let  data_len = meta.query_advice(phase_2_column, Rotation(2));

// 5개 컬럼을 RLC로 압축
let  input_exprs = vec![
1.expr(), // is_enabled
1.expr(), // is_final
input_rlc, // input RLC
data_len, // input length
output_rlc, // output RLC
];
let  table_exprs = keccak_table.table_exprs(meta);

input_exprs
.into_iter()
.zip_eq(table_exprs)
.map(|(input, table)| (q.clone() * input, table))
.collect::<Vec<_>>()
```

```
});
```

Phase별 Challenge 사용

Halo2의 multi-phase 시스템을 활용하여 RLC를 단계적으로 적용합니다.

Phase 구조

```
Phase 1 (FirstPhase)
↓
기본 제약 조건 구성
↓
Phase 2 (SecondPhase)
↓
evm_word, keccak_input challenge 사용 가능
↓
RLC 기반 제약 조건 구성
↓
Phase 3 (ThirdPhase) - SuperCircuit만
↓
lookup_input challenge 사용 가능
↓
최종 룩업 제약 조건
```

Phase 2 컬럼

RLC를 사용하는 컬럼은 Phase 2에 배치됩니다:

파일: `aggregator/src/aggregation/rlc/config.rs`

```rust
#[derive(Debug, Clone, Copy)]
pub struct RlcConfig {
pub(crate) phase_2_column: Column<Advice>,
pub(crate) selector: Selector,
// ...
}

impl RlcConfig {
pub(crate) fn configure(
meta: &mut ConstraintSystem<Fr>,
keccak_table: &KeccakTable,
challenge: Challenges,
) -> Self {
// Phase 2 컬럼 생성
let phase_2_column = meta.advice_column_in(SecondPhase);
meta.enable_equality(phase_2_column);
let challenge_expr = challenge.exprs(meta);
// RLC 게이트 구성
meta.create_gate("rlc_gate", |meta| {
```

```
let a = meta.query_advice(phase_2_column, Rotation(0));
let b = meta.query_advice(phase_2_column, Rotation(1));
let c = meta.query_advice(phase_2_column, Rotation(2));
let d = meta.query_advice(phase_2_column, Rotation(3));
let q1 = meta.query_selector(selector);
// 제약: q1*(a*b+c-d) = 0
// 즉, d = a*b + c (RLC 누적)
let cs1 = q1 * (a.clone() * b + c - d);
// Challenge 추출 제약
let q2 = meta.query_selector(enable_challenge1);
let cs2 = q2 * (a.expr() - challenge_expr.keccak_input());
vec![cs1, cs2]
});
Self { phase_2_column, selector, /* ... */ }
}
}
```

EVM Circuit의 Phase 사용

EVM Circuit은 Phase 1과 Phase 2 컬럼을 모두 사용합니다:

파일: `zkevm-circuits/src/evm_circuit/util.rs`

```
// Phase별 Storage 타입
pub(crate) enum CellType {
```

```
StoragePhase1, // Phase 1 일반 저장소
StoragePhase2, // Phase 2 일반 저장소
StoragePermutation, // Phase 1 복사 제약 가능
StoragePermutationPhase2, // Phase 2 복사 제약 가능
// ...
}

impl CellType {
// Expression의 phase 결정
fn  expr_phase<F: Field>(expr: &Expression<F>) -> u8 {
use Expression::*;
match  expr {
Challenge(challenge) => challenge.phase() + 1,
Advice(query) => query.phase(),
Constant(_) | Selector(_) | Fixed(_) | Instance(_) => 0,
Negated(a) | Scaled(a, _) => Self::expr_phase(a),
Sum(a, b) | Product(a, b) =>
std::cmp::max(Self::expr_phase(a), Self::expr_phase(b)),
}
}

// Expression에 맞는 Storage 타입 반환
pub(crate) fn  storage_for_expr<F: Field>(expr: &Expression<F>) -> CellType {
```

```
Self::storage_for_phase(Self::expr_phase::<F>(expr))
}
pub(crate) fn  storage_for_phase(phase: u8) -> CellType {
match  phase {
0 => CellType::StoragePhase1,
1 => CellType::StoragePhase2,
_ => unreachable!(),
}
}
}
```

RLC의 보안 속성

충돌 확률

두 개의 서로 다른 입력 $(v_0, v_1, \ldots, v_{n-1})$ 과 $(w_0, w_1, \ldots, w_{n-1})$ 이 같은 RLC 값을 가질 확률은 매우 낮습니다:

$$P(RLC(v) = RLC(w)) \leq n / |F|$$

여기서:

- n : 입력 길이
- $|F|$: 필드 크기 (BN254의 경우 약 2^{254})

예를 들어, 32바이트 입력의 경우:

$$P(collision) \leq 32 / 2^{254} \approx 2^{-249}$$

이는 실질적으로 무시할 수 있는 확률입니다.

Schwartz-Zippel Lemma

RLC의 보안성은 Schwartz-Zippel lemma에 기반합니다:

```
다항식 P(x)가 0이 아니고 차수가 d일 때,

무작위 r에 대해 P(r) = 0일 확률은 최대 d/|F|
```

RLC는 차수 n-1의 다항식이므로, 서로 다른 입력이 같은 값을 가질 확률은 (n-1)/|F|로 제한됩니다.

최적화 기법

1. Horner's Method

RLC 계산은 Horner's method를 사용하여 효율적으로 수행됩니다:

```
// 일반적인 방법: v₀ + v₁ ·r + v₂ ·r² + v₃ ·r³

// 필요한 곱셈: 6번 (r², r³ 계산 + 3번의 곱셈)



// Horner's method: ((v₃ ·r + v₂ )·r + v₁ )·r + v₀

// 필요한 곱셈: 3번

fn  generic<V, I>(values: I, randomness: V) -> V

where

I: IntoIterator<Item = V>,

<I as IntoIterator>::IntoIter: DoubleEndedIterator,
```

```
V: Clone + Add<Output = V> + Mul<Output = V>,
{
let mut values = values.into_iter().rev();
let init = values.next().expect("values should not be empty");
values.fold(init, |acc, value| acc * randomness.clone() + value)
}
```

장점:

- 곱셈 횟수 최소화: O(n) 곱셈만 필요
- 거듭제곱 계산 불필요
- 메모리 효율적: 누적 값 하나만 유지

2. Big-Endian vs Little-Endian

데이터 타입에 따라 적절한 바이트 순서를 선택합니다:

```
// Little-endian (EVM word)
pub fn word_rlc(&self, n: U256) -> Value<F> {
self.challenges
.evm_word()
.map(|r| rlc::value(&n.to_le_bytes(), r))
}

// Big-endian (Keccak 입력, 주소 등)
pub(crate) fn rlc_be_bytes<F: Field>(bytes: &[u8], rand: Value<F>) -> Value<F>
```

```
rand.map(|rand| {
bytes
.iter()
.fold(F::zero(), |acc, byte| {
acc * rand + F::from(*byte as u64)
})
})
}
```

선택 기준:

- **Little-endian**: EVM 내부 표현 (워드, 스택 값)
- **Big-endian**: 네트워크 표준 (주소, 해시, 서명)

3. 미리 계산된 거듭제곱

자주 사용되는 거듭제곱은 미리 계산하여 재사용합니다:

```
// Access List에서 r²⁰과 r³² 미리 계산
let r20 = challenges.keccak_input().map(|f| f.pow([20, 0, 0, 0]));
let r32 = challenges.keccak_input().map(|f| f.pow([32, 0, 0, 0]));

for al in access_list.iter() {
// 주소 (20바이트)
section_rlc = section_rlc * r20 + address_rlc;
for sk in al.storage_keys.iter() {
```

```
// 스토리지 키 (32바이트)
section_rlc = section_rlc * r32 + storage_key_rlc;
}
}
```

4. 조건부 RLC 업데이트

불필요한 RLC 업데이트를 건너뛰어 제약 조건을 줄입니다:

```
// Copy Circuit에서 패딩/마스크 시 RLC 업데이트 생략
let  value_acc_next = select::expr(
is_pad.expr() + is_mask.expr(), // 조건
value_acc.expr(), // 조건이 참이면 유지
value_acc.expr() * challenges.keccak_input() + value.expr(), // 업데이트
);
```

파일: `zkevm-circuits/src/keccak_circuit.rs`

```
// Keccak Circuit에서 패딩 바이트 처리
let  new_data_rlc = select::expr(
is_padding.expr(),
new_data_rlc.clone(), // 패딩이면 유지
new_data_rlc.clone() * challenges.keccak_input() + byte.expr.clone(),
);
```

5. 배치 RLC 계산

여러 값의 RLC를 한 번에 계산하여 효율성을 높입니다:

```
// 서명 검증에서 challenge 거듭제곱 배치 생성
let keccak_challenge_powers = iter::successors(
Some(Value::known(F::one())),
|coeff| Some(challenges.keccak_input() * coeff)
)
.take(64)
.collect::<Vec<_>>();

// 배치로 사용
for (i, power) in keccak_challenge_powers.iter().enumerate() {
// power를 사용한 RLC 계산
}
```

실전 사용 패턴

패턴 1: 단순 바이트 배열 인코딩

```
// 바이트 배열을 하나의 필드 원소로 압축
fn encode_bytes<F: Field>(bytes: &[u8], challenge: Value<F>) -> Value<F> {
challenge.map(|r| rlc::value(bytes.iter(), r))
```

```
}

// 사용 예시
let hash_rlc = encode_bytes(&hash_bytes, challenges.keccak_input());
```

패턴 2: 누적 RLC

```
// 스트리밍 방식으로 RLC 누적
let mut rlc_acc = Value::known(F::zero());

for byte in data.iter() {
rlc_acc = rlc_acc
.zip(challenges.keccak_input())
.map(|(acc, r)| acc * r + F::from(*byte as u64));
}
```

패턴 3: 다중 필드 RLC

```
// 여러 필드를 하나의 RLC로 결합
fn encode_multi_field<F: Field>(
field1: Value<F>,
field2: Value<F>,
field3: Value<F>,
```

```
challenge: Value<F>,
) -> Value<F> {
challenge.map(|r| {
field1.map(|f1| {
field2.map(|f2| {
field3.map(|f3| {
(f1 * r + f2) * r + f3
})
})
})
}).flatten().flatten().flatten()
}
```

패턴 4: 제약 조건에서의 RLC

```
// ConstraintBuilder에서 RLC 제약 조건 구성
impl<F: Field> ConstraintBuilder<F> {
pub fn require_equal_word(&mut self, name: &str, lhs: Word<F>, rhs: Word<F>)
// Word는 내부적으로 RLC Expression을 가짐
self.require_equal(name, lhs.expr(), rhs.expr());
}
}
```

```
// 사용 예시
cb.require_equal_word(
"stack value matches",
stack_value,
expected_value,
);
```

패턴 5: 테이블 룩업 RLC

```
// 여러 컬럼을 RLC로 압축하여 룩업
pub fn lookup_with_rlc<F: Field>(
meta: &mut ConstraintSystem<F>,
name: &str,
input_columns: Vec<Expression<F>>,
table_columns: Vec<Expression<F>>,
challenge: Expression<F>,
) {
meta.lookup(name, |meta| {
// 입력 컬럼들을 RLC로 압축
let input_rlc = rlc::expr(&input_columns, challenge.clone());
// 테이블 컬럼들을 RLC로 압축
let table_rlc = rlc::expr(&table_columns, challenge);
vec![(input_rlc, table_rlc)]
```

```
});

}
```

디버깅 및 검증

Mock Challenge 값

테스트 시 고정된 challenge 값을 사용할 수 있습니다:

파일: `zkevm-circuits/src/util.rs`

```rust
#[derive(Default, Clone, Copy, Debug)]
pub struct MockChallenges {
evm_word: u64,
keccak_input: u64,
lookup_input: Option<u64>,
}

impl MockChallenges {
pub fn construct<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
evm_word: 0x100, // 256
keccak_input: 0x101, // 257
lookup_input: Some(0x100),
}
```

```
}
pub fn values<F: Field>(&self, _layouter: &impl Layouter<F>)
-> Challenges<Value<F>>
{
Challenges {
evm_word: Value::known(F::from(self.evm_word)),
keccak_input: Value::known(F::from(self.keccak_input)),
lookup_input: self.lookup_input.map(|c| Value::known(F::from(c))),
}
}
}
```

RLC 값 검증

```
#[cfg(test)]
mod tests {
use super::*;
#[test]
fn test_rlc_calculation() {
let bytes = vec![1u8, 2, 3, 4];
let challenge = Fr::from(256);
// 수동 계산
let expected = Fr::from(1)
```

```
+ Fr::from(2) * challenge

+ Fr::from(3) * challenge * challenge

+ Fr::from(4) * challenge * challenge * challenge;

// RLC 함수 사용

let  result = rlc::value(&bytes, challenge);

assert_eq!(result, expected);

}

}
```

성능 고려사항

제약 조건 수

RLC를 사용하면 다중 컬럼 룩업을 단일 컬럼 룩업으로 줄일 수 있습니다:

```
RLC 없이:

- 5개 컬럼 룩업 = 5개의 제약 조건



RLC 사용:

- 1개 RLC 값 룩업 = 1개의 제약 조건

- RLC 계산 제약 조건 = n개 (n은 입력 길이)



총 제약 조건: 1 + n
```

Trade-off:

- 짧은 입력 (n < 5): RLC가 유리
- 긴 입력 (n > 5): 경우에 따라 다름
- 재사용되는 RLC: 항상 유리 (한 번 계산, 여러 번 사용)

증명 크기

RLC 사용 시 증명 크기 영향:

1. **Phase 2 컬럼 추가**: 약간의 증명 크기 증가
2. **제약 조건 감소**: 증명 크기 감소
3. **전체 효과**: 대부분의 경우 증명 크기 감소

증명 생성 시간

```
RLC 계산 비용:
- Expression 구성: O(n) 시간
- Witness 할당: O(n) 시간
- 제약 조건 검증: O(1) 시간 (룩업당)

전통적 방법:
- 제약 조건 검증: O(k) 시간 (k개 컬럼)

RLC가 유리한 경우:
- 여러 번 룩업되는 경우
- 긴 데이터 (n이 큰 경우)
```

일반적인 함정과 해결책

함정 1: Phase 순서 오류

```
// ❌ 잘못된 예: Phase 1에서 challenge 사용
let cell = meta.advice_column(); // Phase 1
meta.create_gate("wrong", |meta| {
let value = meta.query_advice(cell, Rotation::cur());
let challenge = meta.query_challenge(challenges.evm_word); // Phase 2
vec![value - challenge] // 오류!
});

// ✅ 올바른 예: Phase 2 컬럼 사용
let cell = meta.advice_column_in(SecondPhase);
meta.create_gate("correct", |meta| {
let value = meta.query_advice(cell, Rotation::cur());
let challenge = meta.query_challenge(challenges.evm_word);
vec![value - challenge] // 정상
});
```

함정 2: 바이트 순서 불일치

```
// ❌ 잘못된 예: 순서 불일치
let rlc1 = rlc::value(&bytes, r); // Little-endian
```

```
let rlc2 = rlc_be_bytes(&bytes, r); // Big-endian
// rlc1 ≠ rlc2


// ✅ 올바른 예: 일관된 순서 사용
let rlc1 = rlc::value(&bytes, r);
let rlc2 = rlc::value(&bytes, r);
// rlc1 == rlc2
```

함정 3: 빈 입력 처리

```
// ❌ 잘못된 예: 빈 입력 시 패닉
pub(crate) fn expr<F: Field, E: Expr<F>>(
expressions: &[E],
randomness: E
) -> Expression<F> {
generic(expressions.iter().map(|e| e.expr()), randomness.expr())
// expressions가 비어있으면 패닉!
}


// ✅ 올바른 예: 빈 입력 처리
pub(crate) fn expr<F: Field, E: Expr<F>>(
expressions: &[E],
randomness: E
```

```
) -> Expression<F> {
if !expressions.is_empty() {
generic(expressions.iter().map(|e| e.expr()), randomness.expr())
} else {
0.expr() // 빈 입력은 0
}
}
```

함정 4: Challenge 재사용

```
// ❌ 잘못된 예: 같은 challenge를 다른 용도로 사용
let  word_rlc = rlc::value(&word_bytes, challenges.evm_word());
let  hash_rlc = rlc::value(&hash_bytes, challenges.evm_word());
// 보안 문제 가능성

// ✅ 올바른 예: 용도별 challenge 사용
let  word_rlc = rlc::value(&word_bytes, challenges.evm_word());
let  hash_rlc = rlc::value(&hash_bytes, challenges.keccak_input());
// 명확한 분리
```

고급 기법

1. 계층적 RLC

큰 데이터를 청크로 나누어 계층적으로 RLC를 계산합니다:

```
// 레벨 1: 각 청크의 RLC
let chunk_rlcs: Vec<Value<F>> = data
.chunks(32)
.map(|chunk| {
challenges.keccak_input()
.map(|r| rlc::value(chunk.iter(), r))
})
.collect();


// 레벨 2: 청크 RLC들의 RLC
let r32 = challenges.keccak_input().map(|r| r.pow([32, 0, 0, 0]));
let final_rlc = chunk_rlcs.iter().fold(
Value::known(F::zero()),
|acc, chunk_rlc| {
acc.zip(r32).zip(*chunk_rlc)
.map(|((a, r), c)| a * r + c)
}
);
```

2. 조건부 Challenge 선택

상황에 따라 다른 challenge를 사용합니다:

```
// Public Input Circuit에서 필드 타입에 따른 challenge 선택
let is_rlc_keccak = meta.query_fixed(is_rlc_keccak, Rotation::next());
let rlc_rand = is_rlc_keccak.clone() * challenges.keccak_input()
+ (1.expr() - is_rlc_keccak) * challenges.evm_word();


// 동적으로 선택된 challenge 사용
let rlc_next = rlc_cur * rlc_rand + byte_next;
```

3. RLC 캐싱

자주 사용되는 RLC 값을 캐싱합니다:

```
pub struct RlcCache<F> {
cache: HashMap<Vec<u8>, Value<F>>,
challenge: Value<F>,
}


impl<F: Field> RlcCache<F> {
pub fn get_or_compute(&mut self, bytes: &[u8]) -> Value<F> {
if let Some(cached) = self.cache.get(bytes) {
*cached
} else {
let rlc = self.challenge.map(|r| rlc::value(bytes.iter(), r));
self.cache.insert(bytes.to_vec(), rlc);
```

```
rlc

}

}

}
```

회로별 RLC 사용 요약

| 회로 | Challenge | 용도 | 특징 |

|------|-----------|------|------|

| **EVM Circuit** | `evm_word` | 32바이트 워드 인코딩 | Little-endian, Word 타입 |

| **State Circuit** | `evm_word` | MPT 증명 데이터 | RW 테이블 값 검증 |

| **Bytecode Circuit** | `evm_word` | 코드 해시 | Poseidon 사용 시 고정값 |

| **Keccak Circuit** | `keccak_input` | 입력/출력 데이터 | Big-endian, 스트리밍 |

| **Tx Circuit** | `keccak_input` | CallData, Access List | 섹션별 RLC 누적 |

| **Sig Circuit** | `keccak_input` | 메시지 해시 | 64바이트 메시지 |

| **Copy Circuit** | `keccak_input` | 복사 데이터 추적 | 조건부 업데이트 |

| **PI Circuit** | 둘 다 | Public Input | 필드 타입별 선택 |

| **SHA256 Circuit** | `keccak_input` | 입력 데이터 | 프리컴파일 |

| **ECC Circuit** | `keccak_input` | 페어링 데이터 | 192바이트 입력 |

모범 사례

1. 적절한 Challenge 선택

```rust
// ✅ 좋은 예: 용도에 맞는 challenge
let word_rlc = rlc::value(&word.to_le_bytes(), challenges.evm_word());
let hash_rlc = rlc::value(&hash, challenges.keccak_input());

// ❌ 나쁜 예: 무분별한 challenge 사용
let everything_rlc = rlc::value(&data, challenges.evm_word());
```

2. 바이트 순서 일관성

```rust
// ✅ 좋은 예: 명확한 바이트 순서
// EVM 내부 데이터: Little-endian
let stack_value_rlc = rlc::value(&value.to_le_bytes(), r);

// 네트워크 데이터: Big-endian
let address_rlc = rlc_be_bytes(&address.to_fixed_bytes(), r);

// ❌ 나쁜 예: 혼재된 바이트 순서
let mixed_rlc = rlc::value(&some_bytes, r); // 순서 불명확
```

3. Phase 관리

```rust
// ✅ 좋은 예: Phase 2 컬럼에 RLC 저장
let rlc_column = meta.advice_column_in(SecondPhase);
```

```
meta.enable_equality(rlc_column);


// RLC 값 할당
region.assign_advice(
|| "rlc value",
rlc_column,
offset,
|| challenges.keccak_input().map(|r| rlc::value(&bytes, r))
)?;


// ❌ 나쁜 예: Phase 1 컬럼에 RLC 저장 시도
let wrong_column = meta.advice_column(); // Phase 1
// Challenge를 사용한 값을 할당할 수 없음
```

4. 에러 처리

```
// ✅ 좋은 예: 빈 입력 처리
pub fn safe_rlc<F: Field>(bytes: &[u8], challenge: Value<F>) -> Value<F> {
if bytes.is_empty() {
Value::known(F::zero())
} else {
challenge.map(|r| rlc::value(bytes.iter(), r))
}
```

```
}

// ✅ 좋은 예: 길이 검증
pub fn bounded_rlc<F: Field>(
bytes: &[u8],
challenge: Value<F>,
max_len: usize
) -> Result<Value<F>, Error> {
if bytes.len() > max_len {
return Err(Error::Synthesis);
}
Ok(challenge.map(|r| rlc::value(bytes.iter(), r)))
}
```

5. 문서화

```
// ✅ 좋은 예: 명확한 문서화
/// Computes RLC of transaction CallData
///
/// # Arguments
/// * `call_data` - Transaction call data bytes
/// * `challenge` - Keccak input challenge (big-endian encoding)
///
```

```
/// # Returns

/// RLC value: data[0] + data[1]*r + data[2]*r² + ...

pub fn call_data_rlc<F: Field>(

call_data: &[u8],

challenge: Value<F>

) -> Value<F> {

challenge.map(|r| rlc::value(call_data.iter(), r))

}
```

결론

RLC(Random Linear Combination)는 ZKEVM 구현체의 핵심 최적화 기법으로, 다음과 같은 이점을 제공합니다:

주요 이점

1. **효율적인 데이터 압축**: 여러 바이트를 하나의 필드 원소로 결합
2. **다중 컬럼 룩업**: 여러 컬럼을 하나의 값으로 압축하여 제약 조건 감소
3. **높은 보안성**: Schwartz-Zippel lemma에 기반한 충돌 저항성
4. **유연한 적용**: 다양한 데이터 타입과 회로에 적용 가능

구현 요점

- **Challenge 시스템**: 세 가지 challenge (`evm_word` , `keccak_input` , `lookup_input`)를 용도별로 사용
- **Phase 관리**: Multi-phase 시스템을 활용하여 RLC를 단계적으로 적용

- **Horner's Method**: 효율적인 RLC 계산을 위한 알고리즘
- **바이트 순서**: Little-endian (EVM 내부)과 Big-endian (네트워크 표준) 구분

성능 영향

- **제약 조건 감소**: 다중 컬럼 룩업을 단일 제약 조건으로 축소
- **증명 크기 감소**: 전체적으로 증명 크기 감소 효과
- **계산 효율성**: Horner's method를 통한 최적화된 계산

사용 시 주의사항

1. 적절한 Challenge 선택
2. Phase 순서 준수
3. 바이트 순서 일관성 유지
4. 빈 입력 처리
5. Challenge 재사용 방지

RLC는 ZKEVM의 효율성과 확장성을 가능하게 하는 핵심 기술이며, 올바르게 사용하면 회로 크기와 증명 생성 시간을 크게 줄일 수 있습니다.

관련 문서

- 05-테이블-시스템/09-Lookup-메커니즘.md - 룩업에서의 RLC 활용
- 02-핵심-회로/01-EVM-Circuit.md - EVM Circuit의 Word RLC
- 02-핵심-회로/04-Keccak-Circuit.md - Keccak 입력 RLC
- 02-핵심-회로/05-Tx-Circuit.md - 트랜잭션 데이터 RLC
- 08-최적화-기법/01-CellManager.md - Phase별 셀 관리